Studio Nx

Backend(FastAPI/Python) 06 — get_current_user()

2026-10-12

Backend(FastAPI/Python) 06 — `get_current_user()`

항목	내용
파일	`studionx/backend/app/api/deps.py`
작업	P1-03 백엔드 공통: 로그인한 사용자 찾기 (PRD 01 FR-105, FR-108)
작성	Claude (AI), 2026-10-06, 사용자 검토
이전 · 다음	05 `decode_access_token()` · 07 `get_current_user_optional()`

1. 하는 일

요청 헤더의 Access Token으로 **요청한 회원을 DB에서 찾아** 라우터에 넘깁니다. 토큰이 없거나, 틀렸거나, 회원이 활동 중이 아니면 401 오류를 던집니다.

`Authorization: Bearer eyJhbGciOi…` → `User(user_id=7, …)`

2. 언제 쓰나

로그인이 필요한 API가 함수 인자에 `user: CurrentUser`라고 적어 두면, FastAPI가 라우터보다 **먼저** 이 함수를 부릅니다. 그래서 라우터는 검사가 끝난 회원만 받습니다.

```
@router.get("/users/me")
def read_me(user: CurrentUser) -> MeResponse:   # user = 이 함수가 돌려준 회원
    ...
```

이 함수를 쓰는 API (지금 뼈대 기준)	화면
`GET /users/me`, `PATCH /users/me`, 프로필 사진, 비밀번호 변경, 탈퇴	userInfo.html
`GET /me/dashboard`	myPage.html
`GET`, `POST`, `PATCH`, `DELETE /projects…`	myProject.html, studioNx.html
쓰지 않는 API: `POST /inquiries`	로그인이 선택이라서 07 `get_current_user_optional()`을 씁니다

순서	일어나는 일
① 브라우저	`GET /api/v1/me/dashboard`를 `Authorization: Bearer eyJ…`와 함께 보냅니다 (JS 02)
② `bearer_scheme`	헤더에서 `eyJ…`만 꺼냅니다
③ `get_db`	DB 세션을 엽니다
④ **이 함수**	토큰을 검사하고(05) 7을 얻은 뒤, DB에서 7번 회원을 찾아 상태를 확인합니다
⑤ 라우터	`user`로 7번 회원을 받아 대시보드를 만듭니다
실패하면	라우터는 실행되지 않습니다. 401로 응답하면 브라우저의 `api.js`가 재발급을 받은 뒤 한 번 다시 보냅니다

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import Depends
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from sqlalchemy.orm import Session

from app.core.errors import AppError, ErrorCode
from app.core.security import decode_access_token
from app.db.session import get_db
from app.models import User
```

`deps.py` 맨 위의 import입니다. **`AppError`, `ErrorCode`, `decode_access_token` 두 줄을 이번에 추가했습니다.**

import	가져오는 것	쓰는 곳
`Annotated` (파이썬 기본)	타입에 덧붙임 정보를 다는 도구	`Bearer`, `Db`, `CurrentUser` 별칭
`Depends`	"이 값은 이 함수로 만들어 넣어라"라는 표시	별칭 안
`HTTPBearer`, `HTTPAuthorizationCredentials`	`Authorization: Bearer …` 헤더를 읽는 도구와 그 결과	`bearer_scheme`, `credentials`
`Session`	DB 세션 타입 (SQLAlchemy)	`Db`
`AppError`, `ErrorCode`	01번의 오류 클래스와 코드	401
`decode_access_token`	05번의 토큰 검사	2단계
`get_db`	요청마다 DB 세션을 열고 닫는 함수	`Db`
`User`	users 테이블 모델	`db.get(User, …)`

함수 (이미 있던 별칭 포함)

```
bearer_scheme = HTTPBearer(auto_error=False)
Bearer = Annotated[HTTPAuthorizationCredentials | None, Depends(bearer_scheme)]
Db = Annotated[Session, Depends(get_db)]


def get_current_user(
    credentials: Bearer,
    db: Db,
) -> User:
    if credentials is None:
        raise AppError(ErrorCode.UNAUTHORIZED, 401, "Please sign in.")
    user_id = decode_access_token(credentials.credentials)
    user = db.get(User, user_id)
    if user is None or user.status != "ACTIVE":
        raise AppError(ErrorCode.UNAUTHORIZED, 401, "Please sign in again.")
    return user


CurrentUser = Annotated[User, Depends(get_current_user)]
```

문법 풀이

import 문법	뜻
`from 모듈 import 이름`	모듈 안의 `이름`만 가져와서 그대로 씁니다
`app.…`	이 프로젝트(`studionx/backend/app/`)의 파일입니다. `app.api.deps`는 `app/api/deps.py`입니다

식	뜻
`Depends(함수)`	FastAPI가 요청마다 그 함수를 먼저 실행해서 결과를 인자로 넣어 줍니다 ("의존성")
`Annotated[타입, Depends(…)]`	타입과 `Depends`를 한 이름으로 묶습니다. 그래서 `user: CurrentUser` 한 번만 쓰면 됩니다
`HTTPBearer(auto_error=False)`	헤더가 없거나 `Bearer`로 시작하지 않으면 **오류 대신 `None`**을 넣어 줍니다. 기본값(`True`)이라면 FastAPI가 그 자리에서 401 오류를 던집니다
`credentials.credentials`	`Bearer eyJ…`에서 `eyJ…` 부분입니다 (`credentials.scheme`은 `"Bearer"`입니다)
`db.get(User, 7)`	기본 키(`user_id`)가 7인 행을 찾습니다. 없으면 `None`입니다
`A is None or B`	`A`가 참이면 `B`를 보지 않습니다. 그래서 `None.status` 오류가 나지 않습니다

요청	결과
헤더 없음	401 "Please sign in."
`Authorization: Basic …`	401 "Please sign in." (`Bearer`가 아니라서 `None`이 됩니다)
토큰이 틀리거나 만료됨	401 "Please sign in again." (05번이 던집니다)
토큰은 맞는데 회원이 없음	401 "Please sign in again."
회원의 `status`가 `SUSPENDED` 또는 `DELETED`	401 "Please sign in again."
회원의 `status`가 `ACTIVE`	그 회원(`User`)을 돌려줍니다

4. 흐름

1. 헤더가 없으면 401 "Please sign in."으로 응답합니다.
2. 토큰을 검사해서 `user_id`를 얻습니다 (05). 실패하면 401로 응답합니다.
3. DB에서 그 회원을 찾습니다.
4. 회원이 없거나 `ACTIVE`가 아니면 401 "Please sign in again."으로 응답합니다.
5. 회원을 돌려줍니다.

5. 왜 이렇게

선택	이유
토큰만 믿지 않고 DB를 한 번 더 확인	Access Token은 30분 동안 유효합니다. 그사이 탈퇴(`DELETED`)하거나 정지(`SUSPENDED`)된 회원의 토큰을 막기 위해서입니다 (FR-108)
없는 회원, 정지, 탈퇴를 같은 401과 같은 문구로 처리	05번과 같은 이유입니다. 이유를 알려 주면 "이 번호의 회원은 탈퇴했다" 같은 정보가 새어 나갑니다
403이 아니라 401로 응답	프론트 `api.js`는 401 `UNAUTHORIZED`일 때만 재발급을 시도하고, 실패하면 로그인 화면으로 보냅니다 (JS 02). 정지되거나 탈퇴한 회원은 재발급도 거절되므로(P1-05) 결국 로그인 화면으로 갑니다

선택	이유
문구를 두 가지로 나눔	헤더가 없으면 처음부터 로그인하지 않은 것이므로 "Please sign in."이고, 토큰이 있는데 안 되면 "Please sign in again."입니다
`auto_error=False`로 두고 직접 401을 던짐	같은 `Bearer`를 07번 `get_current_user_optional()`도 씁니다. 07번은 헤더가 없을 때 오류가 아니라 `None`(손님)으로 처리해야 합니다. 그래서 헤더 읽기는 오류를 내지 않게 두고, 판단은 각 함수가 합니다
`status != "ACTIVE"`를 문자열로 비교	users 모델의 `status` 칸은 이 작업의 다음 순서(모델 컬럼)에서 만듭니다. 그때 `StrEnum`으로 만들면 `UserStatus.ACTIVE == "ACTIVE"`가 참이므로, 이 코드는 그대로 동작합니다

6. 시험 (`tests/test_deps.py` 새 파일, 7개)

users 모델의 컬럼(`status` 등)이 아직 없어서, 진짜 DB 대신 **가짜 세션(`FakeDb`)**을 넣었습니다. `FakeDb`는 `db.get(User, id)`만 흉내 내고, 받은 인자를 기록합니다. 시험용 앱에 `GET /whoami`(`user: CurrentUser` → `{"user_id": …}`)를 만들어 요청했습니다.

경우	결과
정상 토큰과 `ACTIVE` 회원 7	200 `{"user_id": 7}`이고, DB에 `(User, 7)`로 한 번 물어봅니다
헤더 없음	401 "Please sign in."이고, DB에 묻지 않습니다
`Authorization: Basic …`	401 "Please sign in."
틀린 토큰	401 "Please sign in again."이고, DB에 묻지 않습니다
토큰의 회원 99가 DB에 없음	401 "Please sign in again."
`SUSPENDED`, `DELETED` 회원 (2개)	401 "Please sign in again."

`app.dependency_overrides[get_db] = …`는 시험에서 `get_db` 대신 다른 함수를 쓰게 하는 FastAPI 기능입니다.

전체 시험 42개가 통과했고, `ruff` 검사도 통과했습니다.